

在 Cloud Run 建構及部署寵物護照代理

來源：<https://codelabs.developers.google.com/next26/build-adk-agent-google-mcps?hl=zh-tw>

步驟數：13

瞭解如何使用 ADK 和 Google MCP 伺服器，在 Cloud Run 建構及部署寵物護照 AI 代理

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [安裝](#)
4. [取得 API 金鑰](#)
5. [安裝依附元件](#)
6. [驗證 MCP 伺服器](#)
7. [建立 BigQuery 資料表](#)
8. [連線至 MCP 伺服器](#)
9. [建立代理程式](#)
10. [在本機執行應用程式](#)
11. [部署至 Cloud Run](#)
12. [提示範例](#)
13. [清除所用資源](#)

1. 總覽

在本程式碼研究室中，您將瞭解如何部署 **Pet Passport** 應用程式。這款 AI 代理會使用 Model Context Protocol (MCP) 整合資料分析和定位服務。

這款應用程式會根據紐約市的犬種熱門程度，協助使用者規劃與愛犬共度的完美一天。代理會使用「巨觀到微觀」的推理鏈：

- 策略性探索 (BigQuery)**：找出特定品種人口數最高的紐約市郵遞區號。
- 本地執行 (地圖)**：使用該郵遞區號做為位置偏誤，尋找「寵物友善咖啡廳」和「狗公園」。
- 行程生成**：結合資料來建立「寵物護照」行程，並附上可點選的連結和圖片。

這個代理是使用 `google-adk` 框架建構而成，並由 Gemini 提供技術支援。

注意：您可以在 [GitHub](#) 上取得完整專案程式碼，包括前端 UI。在本程式碼研究室中，我們將著重於核心代理程式邏輯和基礎架構設定。

步驟 2 · 約 0 分鐘

2. 設定和需求

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

首先，請確認開發環境設定正確無誤。

1. 使用 Google Cloud 進行驗證

設定有效的 Google Cloud 雲端專案並進行驗證。代理程式必須具備這項權限，才能存取 BigQuery 和其他服務。

```
gcloud config set project [YOUR-PROJECT-ID]
gcloud auth application-default login --project [YOUR-PROJECT-ID]
```

注意：如果在驗證期間遇到與其他專案相關的錯誤，可以停用配額專案並手動設定，藉此略過錯誤：

```
gcloud auth application-default login --disable-quota-project
gcloud auth application-default set-quota-project [YOUR-PROJECT-ID]
```

2. 軟體需求

您必須在本機電腦上安裝下列軟體：

- **Python** (必須是 3.13 以上版本)
- **Git** (用於下載存放區)

下載存放區

您可以在 Google MCP 存放區中找到這個專案的程式碼。複製存放區並前往專案資料夾：



```
git clone https://github.com/google/mcp.git
cd examples/petpassport
```

步驟 3 · 約 0 分鐘

3. 安裝

現在您已取得檔案，接下來請設定 Python 環境。

1. 建立虛擬環境：這樣可確保依附元件彼此獨立。

```
python3 -m venv .venv
```

2. 啟動虛擬環境：

- Linux/macOS：

```
source .venv/bin/activate
```

- Windows 系統：

```
.venv\Scripts\activate
```

3. 安裝依附元件：

```
pip install google-adk==1.28.0 python-dotenv google-genai pillow uvicorn
```

啟用 Cloud API

在專案中啟用下列 API：

```
gcloud services enable \  
  bigquery.googleapis.com \  
  aiplatform.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  run.googleapis.com \  
  storage.googleapis.com
```

選擇區域

在殼層中將區域設為環境變數：

```
export REGION=us-central1
```

步驟 4 · 約 0 分鐘

4. 取得 API 金鑰

如要使用 Maps 和 Gemini 服務，您需要取得 API 金鑰，並將其儲存在專案根目錄的 `.env` 檔案中。

1. Google Maps API 金鑰

1. 前往 [Google Cloud 控制台](#)。
2. 依序前往「APIs & Services」(API 和服務) > 「Credentials」(憑證)。
3. 依序按一下「建立憑證」 > 「API 金鑰」。
4. 複製產生的金鑰，並以 `MAPS_API_KEY=[YOUR_KEY]` 形式新增至 `.env` 檔案。
5. (建議) 限制金鑰，只允許 MCP 伺服器使用的 Maps API。

2. Gemini API 金鑰 (AI Studio)

1. 前往 [Google AI Studio](#)。
 2. 按一下「取得 API 金鑰」或前往「API 金鑰」專區。
 3. 按一下「建立 API 金鑰」。
 4. 複製金鑰，並以 `GEMINI_API_KEY=[YOUR_KEY]` 形式新增至 `.env` 檔案。
-

步驟 5 · 約 0 分鐘

5. 安裝依附元件

在 `petpassport/` 資料夾中建立 `requirements.txt` 檔案：

```
google-adk==1.28.0
python-dotenv
google-genai
pillow
```

6. 驗證 MCP 伺服器

這個應用程式會透過 Model Context Protocol (MCP) 伺服器與 Google 地圖和 BigQuery 互動，如要驗證這些伺服器，您必須設定適當的環境變數和標頭。

1. **Google 地圖 MCP**：必須在 `X-Goog-API-Key` 標頭中傳遞有效的 Maps API 金鑰。
2. **BigQuery MCP**：需要具備 BigQuery 服務存取權的 OAuth 憑證。在 Cloud Run 上執行時，代理程式會使用預設的 Compute 服務帳戶；在本機執行時，則會使用本機憑證。

我們在存放區中提供設定指令碼 `setup/setup_env.sh`，協助您在 `.env` 檔案中設定這些變數。

步驟 7 · 約 0 分鐘

7. 建立 BigQuery 資料表

在代理程式查詢狗牌資料前，我們需要在 BigQuery 中建立資料集和資料表，並載入資料。

我們提供設定指令碼 `setup/setup_bigquery.sh`，可執行下列步驟：

1. 建立名為 `pet-passport-data-[PROJECT_ID]` 的 Cloud Storage bucket，用於儲存原始資料。
2. 下載紐約市犬隻執照公開資料集 (CSV)。
3. 將 CSV 上傳至值區。
4. 建立名為 `nyc_dogs` 的 BigQuery 資料集。
5. 將 bucket 中的資料載入資料集內名為 `licenses` 的資料表。

如要執行設定指令碼，請在終端機中執行下列指令：

```
bash setup/setup_bigquery.sh
```

步驟 8 · 約 0 分鐘

8. 連線至 MCP 伺服器

這個應用程式的重要部分是使用 MCP 連線至資料和服務。在本節中，您將在名為 `petpassport/tools.py` 的檔案中，設定 BigQuery 和 Google 地圖的 MCP 工具集。

完成 `tools.py` 程式碼

以下是 `tools.py` 的完整實作方式，包括 MCP 工具集和圖片與資料持續性的自訂工具。我們已將這個程式碼最佳化，將 bucket 解析度移至模組層級，減少多餘的程式碼：

```

import os
import dotenv
import google.auth
import time
import datetime
from google.cloud import storage
from PIL import Image
from google import genai
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StreamableHTTPConnectionParams

MAPS_MCP_URL = "https://mapstools.googleapis.com/mcp"
BIGQUERY_MCP_URL = "https://bigquery.googleapis.com/mcp"

PROJECT_ID = os.getenv('GOOGLE_CLOUD_PROJECT', 'project_not_set')
BUCKET_NAME = f"pet-passport-data-{PROJECT_ID}"

def get_maps_mcp_toolset():
    dotenv.load_dotenv()
    maps_api_key = os.getenv('MAPS_API_KEY', 'no_api_found')

    tools = MCPToolset(
        connection_params=StreamableHTTPConnectionParams(
            url=MAPS_MCP_URL,
            headers={
                "X-Goog-API-Key": maps_api_key
            },
            timeout=30.0,
            sse_read_timeout=300.0
        )
    )
    print("Maps MCP Toolset configured.")
    return tools

def get_bigquery_mcp_toolset():
    credentials, project_id = google.auth.default(
        scopes=["https://www.googleapis.com/auth/bigquery"]
    )

    credentials.refresh(google.auth.transport.requests.Request())
    oauth_token = credentials.token

    HEADERS_WITH_OAUTH = {
        "Authorization": f"Bearer {oauth_token}",
        "x-goog-user-project": project_id
    }

    tools = MCPToolset(
        connection_params=StreamableHTTPConnectionParams(
            url=BIGQUERY_MCP_URL,
            headers=HEADERS_WITH_OAUTH,

```

```

        timeout=30.0,
        sse_read_timeout=300.0
    )
)
print("BigQuery MCP Toolset configured.")
return tools

def generate_pet_passport_photo(prompt: str, image_path: str = None) -> str:
    """Generates an image using gemini-3.1-flash-image-preview based on a prompt and a
    reference image."""
    client = genai.Client()
    output_path = f"/tmp/pet_passport_{int(time.time())}.png"

    try:
        image = Image.open(image_path)
        response = client.models.generate_content(
            model="gemini-3.1-flash-image-preview",
            contents=[prompt, image],
        )

        for part in response.parts:
            if part.inline_data is not None:
                generated_image = part.as_image()
                generated_image.save(output_path)

                # Upload to GCS and generate signed URL
                try:
                    storage_client = storage.Client()
                    bucket = storage_client.bucket(BUCKET_NAME)
                    blob_name = os.path.basename(output_path)
                    blob = bucket.blob(blob_name)

                    blob.upload_from_filename(output_path)

                    url = blob.generate_signed_url(
                        version="v4",
                        expiration=datetime.timedelta(hours=24),
                        method="GET",
                    )
                    return url
                except Exception as e:
                    print(f"Error uploading image to GCS: {e}")
                    return output_path

            raise ValueError("No image was returned by the model.")
    except Exception as e:
        print(f"Error generating image: {e}")
        raise

def save_pet_passport(user_id: str, breed: str, postal_code: str, route_details: str,
image_paths: list[str] = None) -> str:
    """Appends the generated itinerary to the user's history in GCS."""

```

```
try:
    storage_client = storage.Client()
    bucket = storage_client.bucket(BUCKET_NAME)
    blob = bucket.blob(f"user-{user_id}.json")

    # Download existing or start fresh
    # ... (Implementation details hidden for brevity) ...
    return "Success"
except Exception as e:
    print(f"Error saving path: {e}")
    raise
```

程式碼說明：`tools.py`

- `get_maps_mcp_toolset` 和 `get_bigquery_mcp_toolset` 使用正確的端點和驗證標頭設定 MCP 用戶端。
 - `generate_pet_passport_photo` 會使用 Gemini 建立場景，並將結果上傳至 Google Cloud Storage，然後將經簽署的網址傳回前端，以在伺服器重新啟動後繼續運作。
-

步驟 9 · 約 0 分鐘

9. 建立代理程式

工具設定完成後，現在該來建構代理的「大腦」。您將使用 Agent Development Kit (ADK) 在名為 `petpassport/agent.py` 的檔案中建立代理。

完成 `agent.py` 程式碼

以下是 `agent.py` 的完整實作，我們將定義代理程式及其指令：

```
import os
import dotenv
import tools
from google.adk.agents import LlmAgent

dotenv.load_dotenv()

PROJECT_ID = os.getenv('GOOGLE_CLOUD_PROJECT', 'project_not_set')

maps_toolset = tools.get_maps_mcp_toolset()
bigquery_toolset = tools.get_bigquery_mcp_toolset()

root_agent = LlmAgent(
    model='gemini-2.5-pro',
    name='root_agent',
    instruction=f"""
        You are the Pet Passport Agent. Your goal is to help users find a fun walking route
        for their dog in NYC.

        When given a breed and a postal code, follow this flow:
        1. **Strategic Discovery:** Use BigQuery to find the most popular neighborhood for
        that breed in NYC.
        2. **Local Execution:** Use Maps to build a walking route with specific places
        (parks, cafes) in that area.

        **NO DIRECTIONS LINKS:** You must NOT include a Google Maps directions link (e.g.,
        `https://www.google.com/maps/dir/...`) in your final response. Only provide links to
        individual places.

        After generating the itinerary, you MUST call the `save_pet_passport` tool to save
        this path to the user's profile. Pass a clean summary of the itinerary as `route_details`.
        The summary should include details (like rating, description from maps).
    """,
    tools=[maps_toolset, bigquery_toolset, tools.generate_pet_passport_photo,
           tools.save_pet_passport]
)
```

程式碼說明：`agent.py`

- 我們會直接匯入 `tools` (扁平結構)，以支援容器環境。
- 代理程式會以 `gemini-2.5-pro` 初始化。

- 這些指令定義了嚴格的多步驟思維鏈提示 (先 BigQuery，再 Maps)，並嚴格禁止產生或算繪會導致雜亂的步行路線。
-

步驟 10 · 約 0 分鐘

10. 在本機執行應用程式

部署至 Cloud Run 前，建議先在本機測試應用程式。

1. 確認您位於專案目錄：

```
cd examples/petpassport
```

2. 啟動 **FastAPI** 伺服器：我們使用 `uvicorn` 執行應用程式。進入點是 `petpassport` 資料夾內的 `main.py`。

```
uvicorn petpassport.main:app --reload
```

3. 開啟使用者介面：在瀏覽器中前往 `http://127.0.0.1:8000/ui/`，與寵物護照介面互動。

11. 部署至 Cloud Run

代理程式準備就緒後，就可以部署至 Cloud Run。我們直接使用標準 `gcloud` 指令，嚴格控管容器環境。

在專案目錄中執行下列指令：

```
gcloud run deploy petpassport \
  --source petpassport \
  --region $REGION \
  --allow-unauthenticated \
  --labels dev-tutorial=google-mcp
```

設定環境變數

部署完成後，請前往 Google Cloud 控制台 中的 Cloud Run 服務，並在「變數和密鑰」分頁下方設定下列環境變數：

- `MAPS_API_KEY`：您的 Google Maps API 金鑰。
 - `GOOGLE_CLOUD_PROJECT`：專案 ID。
 - `PROJECT_ID`：您的專案 ID (舊版模組支援備援)。
-

12. 提示範例

嘗試使用下列提示詞與部署的代理互動：

1. **標準：**「我想在紐約市 10021 附近，帶我的黃金獵犬去散步。請幫我們規劃一條有咖啡廳的路線。」
 2. **不同品種：**「我養的是法國鬥牛犬，住在上西城 (10024 附近)。建議一條短程路線，並在熱門的狗狗公園停下來。」
 3. **附上圖片：** (上傳愛犬的相片) 「這是我的柯基犬！我們在 10013 附近。請為我們規劃完美的出遊行程。」
-

步驟 13 · 約 0 分鐘

13. 清除所用資源

如要避免系統收取本教學課程所用資源的費用，請執行下列動作：

- 刪除 Cloud Run 服務：`gcloud run services delete petpassport --region=$REGION`
 - 刪除 GCS bucket：`gcloud storage rm -r gs://pet-passport-data-$PROJECT_ID`
-